



République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Alger 1 – Faculté des sciences.

Département Informatique

M2 ISII : Module : Ingénierie Web
(Enseignante : Madame Guesmia)

Thème :

Architecture du site d'événements

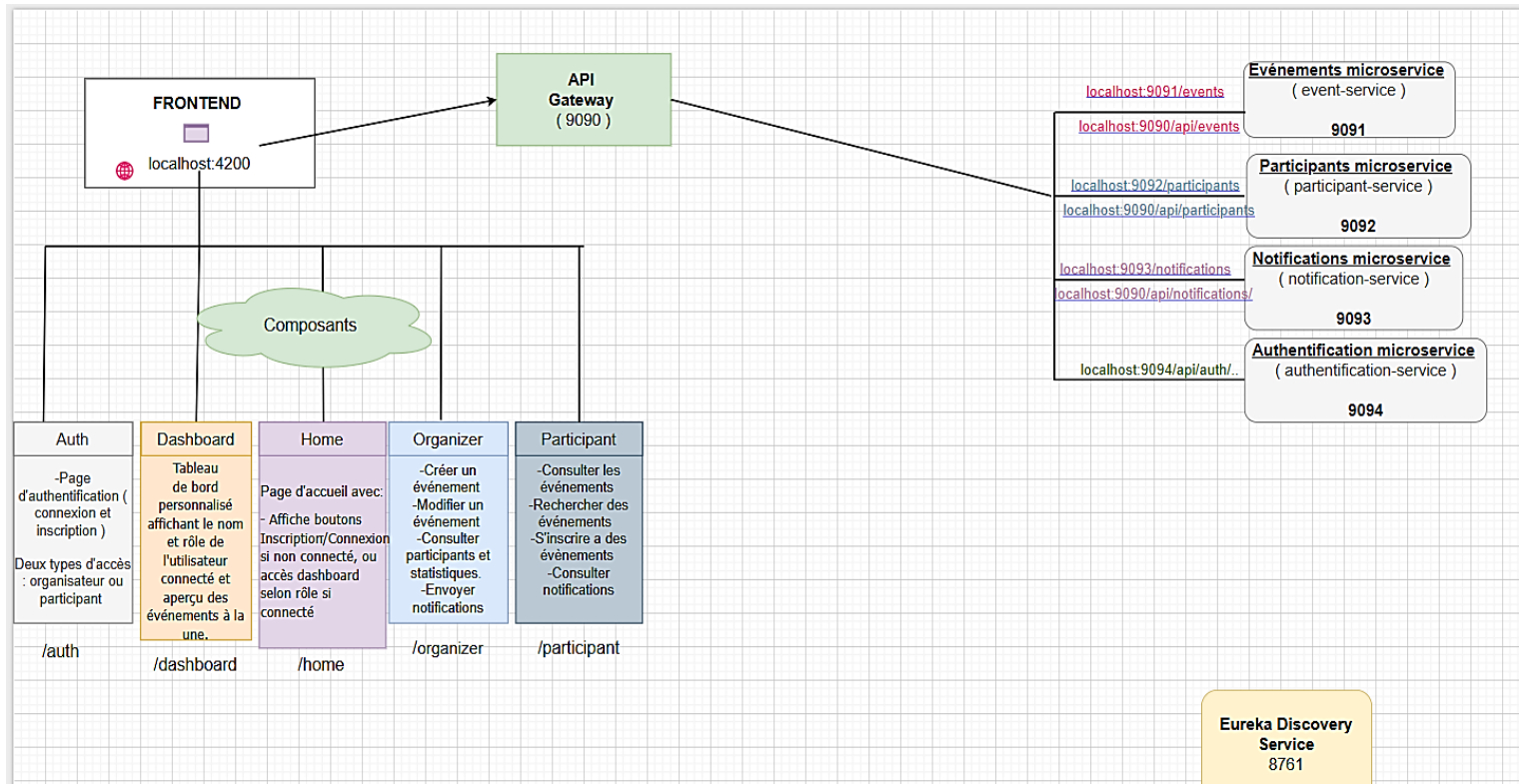
Réalisé par :

-Meradji Anaïs (groupe 3)

Repository github : https://github.com/meradji-anais/gestion_events_webApp

Année universitaire : 2025/2026

Architecture complète :



Backend : les micro-services :

Microservice	Description	Entité(s)	Endpoints REST
Event MicroService (9091)	Gestion des événements et médias associés. (Le formulaire du frontend contiendra un bouton pour ajouter une image .png .jpg ou alors une video .mp4 ou bien un document .pdf , en plus des champs obligatoires tels que date, type, lieu..titre)	Event {id, title, description, type, location, eventDate, capacity, organizerName, organizerEmail, imageUrl, createdAt} Media {id, eventId, type, filename, url, uploadedAt}	Events: GET /api/events (récupérer les events) POST /api/events (ajouter un event) GET /api/events/{id} (récupérer un event) PUT /api/events/{id} (modification event) DELETE /api/events/{id} (suppression event) GET /api/events/organizer/{email} GET /api/events/search GET /api/events/type/{type} (filtrage) GET /api/events/location/{location} GET /api/events/{id}/statistics POST /api/events/{eventId}/upload (image) Médias: POST /api/events/{eventId}/media (ajout) GET /api/events/{eventId}/media GET /api/events/{eventId}/media/{type} GET /api/events/file/{filename} DELETE /api/events/media/{mediaId}
Participant MicroService (9092)	Gestion des inscriptions aux événements (les participants)	Participant {id, eventId, name, email, status, registeredAt}	GET /api/participants POST /api/participants GET /api/participants/event/{eventId} GET /api/participants/email/{email} DELETE /api/participants/{id}
Notification MicroService (9093)	Gestion des notifications et alertes	Notification {id, email, type, message, eventId, isRead, createdAt}	POST /api/notifications (notif auto) GET /api/notifications/{email} (recuperation) GET /api/notifications/{email}/unread GET /api/notifications/{email}/unread/count PUT /api/notifications/{id}/read PUT /api/notifications/{email}/read-all (pour marquer toutes les notifs comme lues) DELETE /api/notifications/{id}

Microservice	Description	Entité(s)	Endpoints REST
Authentification MicroService (9094)	Gestion de l'authentification et des utilisateurs	User {id, email, password, role, name, createdAt}	POST /api/auth/register (inscription) POST /api/auth/login (connexion) POST /api/auth/logout (déconnexion) GET /api/auth/current-user (verification de connexion et récupération d'informations) GET /api/auth/health (juste pour vérifier que le service d'authentification fonctionne)

J'ai également créé des micro services techniques :

Eureka Discovery Service (8761)	Service de découverte et registre des micro-services	/	Endpoints Eureka par défaut
API Gateway (9090)	Routage et point d'entrée unifié pour tous les microservices (pour centraliser les microservices)	/	Routes configurées vers les microservices

Communication entre les microservices :

Communications synchrones (Rest) :

Event - > Participant

- Récupération de la liste des participants pour les statistiques (dans le dashboard organizer)
- Récupération de la liste des participants pour envoyer des notifications

Event - >Notification

- Envoi de notifications lors de la modification d'un événement (auquel le participant est inscrit)
- Envoi de notifications lors de l'annulation d'un événement (auquel le participant est inscrit)

Participant – > Event

- Récupération des détails d'un événement lors de l'inscription (pour avoir le titre)

Participant - > Notification

- Envoi automatique d'une notification de confirmation d'inscription

PS :

NOTIFICATION-SERVICE (9093) communique avec aucun autre microservice car c'est un service passif qui reçoit uniquement des requêtes des autres, il stocke et gère les notifications.

AUTHENTIFICATION-SERVICE (9094) est un service isolée qui gère uniquement l'authentification, les autres services ne communiquent pas avec.

Frontend : les composants web :

Mis à part le composant racine, il y'en a 5.

Composant	Description	Routes
AppComponent	Conteneur principal de l'application avec header, navigation dynamique (selon rôle, participant ou organisateur), menu profil avec déconnexion et router-outlet	/ (racine)
HomeComponent	Page d'accueil avec présentation de l'application et aperçu des 3 prochains événements. - Affiche boutons Inscription/Connexion si non connecté, ou accès dashboard selon rôle si connecté	/ ou /home
AuthComponent	Page d'authentification avec tabs (Connexion/Inscription), formulaires de login et register, sélection de rôle (Organisateur/Participant)	/auth
DashboardComponent	Tableau de bord personnalisé affichant le nom et rôle de l'utilisateur connecté, avec accès direct à son espace (Organisateur ou Participant) et aperçu des événements à la une.	/dashboard (protégé par authGuard)
OrganizerDashboardComponent	<u>Interface organisateur</u> : créer/modifier/supprimer des événements, gérer les participants, consulter statistiques, envoyer notifications (option bonus, car il y'a un mécanisme de notifications automatiques implémenté), uploader médias (images, vidéos, PDFs) lors de l'ajout d'un event (possibilité de modifier le media ou le supprimer)	/organizer (protégé par authGuard + roleGuard)

Composant	Description	Routes
ParticipantDashboardComponent	<u>Interface participant</u> : rechercher des événements (par mot-clé / date), s'inscrire/se désinscrire, consulter ses inscriptions, recevoir et gérer les notifications (les notifications automatiques de modification / suppression d'évènement survenues cotée organisateur, et notifications automatiques de confirmation d'inscription et confirmation de désinscription), et aussi les notifications manuelles que les organisateurs pourront lui envoyer.	/participant (protégé par authGuard + roleGuard)

Protections des routes :

- ✚ **/dashboard** est protégé par authGuard : seuls les utilisateurs connectés peuvent y accéder. Si un utilisateur non connecté tente d'y accéder, il est automatiquement redirigé vers /auth.
- ✚ **/organizer** est protégé par authGuard + roleGuard : seuls les utilisateurs connectés avec le rôle ORGANIZER peuvent y accéder. Si un participant tente d'y accéder, il est redirigé vers /dashboard.
- ✚ **/participant** est protégé par authGuard + roleGuard : seuls les utilisateurs connectés avec le rôle PARTICIPANT peuvent y accéder. Si un organisateur tente d'y accéder, il est redirigé vers /dashboard.

Ces protections garantissent que chaque utilisateur accède uniquement aux fonctionnalités qui correspondent à son rôle dans l'application. (mécanisme de sécurité)

